

Activity 8 - Track stock in a HashMap

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K3/A3

Specification

Inventory::receive(&mut self, sku, qty) -> Option<u32> (None on overflow, unchanged); ship -> Result<u32, String> (unknown SKU or insufficient stock is Err, unchanged); quantity(sku) -> u32 (0 when unknown); low_stock(threshold) -> Vec<(String, u32)> strictly below threshold, sorted by SKU.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain Vec, HashMap, the entry API, Option from get, structs and sorting for deterministic output to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is samples/01_vec_basics.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on Vec, HashMap, the entry API, Option from get, structs and sorting for deterministic output with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles collections (lists and dictionaries) with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: Inventory::receive(&mut self, sku, qty) -> Option<u32> (None on overflow, unchanged); ship -> Result<u32, String> (unknown SKU or insufficient stock is Err, unchanged); quantity(sku) -> u32 (0 when unknown); low_stock(threshold) -> Vec<(String, u32)> strictly below threshold, sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Track stock in a HashMap" in starter/src so that the supplied tests pass. Keep the HashMap private and the method signatures unchanged; no unwrap() on lookups. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_vec_basics.rs

Run: rustc --edition 2021 01_vec_basics.rs && ./01_vec_basics

// Sample: a growable Vec, sorting, safe access with get.

```
fn main() {
    let mut skus: Vec<String> = Vec::new();
    skus.push("CCC-0003".to_string());
    skus.push("AAA-0001".to_string());
    skus.push("BBB-0002".to_string());
    skus.sort();
    println!("sorted: {skus:?}");
    println!("first: {:?}, tenth: {:?}", skus.first(), skus.get(9));
    for (i, sku) in skus.iter().enumerate() {
        println!("{i}. {sku}", i + 1);
    }
}
```

Expected output:

```
sorted: ["AAA-0001", "BBB-0002", "CCC-0003"]
first: Some("AAA-0001"), tenth: None
1. AAA-0001
2. BBB-0002
3. CCC-0003
```

02_hashmap_entry.rs

Run: rustc --edition 2021 02_hashmap_entry.rs && ./02_hashmap_entry

// Sample: counting stock with HashMap::entry, then sorting for stable output.
use std::collections::HashMap;

```
fn main() {
    let deliveries = [("ABC-0001", 5), ("ABC-0002", 3), ("ABC-0001", 2)];
    let mut stock: HashMap<&str, u32> = HashMap::new();
    for (sku, qty) in deliveries {
        *stock.entry(sku).or_insert(0) += qty;
    }
    let mut rows: Vec<(&str, u32)> =
        stock.iter().map(|(s, q)| (*s, *q)).collect();
    rows.sort(); // HashMap order is not defined: sort before printing
    println!("{rows:?}");
    match stock.get("ZZZ-9999") {
        Some(qty) => println!("found {qty}"),
        None => println!("ZZZ-9999 is not stocked"),
    }
}
```

Expected output:

```
[("ABC-0001", 7), ("ABC-0002", 3)]
ZZZ-9999 is not stocked
```

Learn more

Rust Book: Storing Lists of Values with Vectors: <https://doc.rust-lang.org/book/ch08-01-vectors.html>

Rust Book: Storing Keys with Values in Hash Maps:
<https://doc.rust-lang.org/book/ch08-03-hash-maps.html>

Rust Book: Defining and Instantiating Structs:

<https://doc.rust-lang.org/book/ch05-01-defining-structs.html>

W3Schools: Rust Vectors: https://www.w3schools.com/rust/rust_vectors.php

W3Schools: Rust HashMap: https://www.w3schools.com/rust/rust_hashmap.php

W3Schools: Rust Structs: https://www.w3schools.com/rust/rust_structs.php

Programiz: Rust Vector: <https://www.programiz.com/rust/vector>

Programiz: Rust HashMap: <https://www.programiz.com/rust/hashmap>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php